DÉPLOIEMENT D'APPLICATIONS INTELLIGENTES · MLOps

TP 3 — Enregistrer modèles et artefacts dans MLflow

Durée : 3 h · Partie 3 du cours · binôme de 2

Journaliser une métrique ne suffit pas a rendre un modèle exploitable six mois plus tard. Ce TP traite le packaging complet : signature, environnement, artefacts d'évaluation, registre, transitions d'alias et service HTTP du modèle promu. Version Windows : toutes les commandes sont écrites pour PowerShell 5.1 ou 7, livré avec Windows 10 et 11. Ouvrez « Windows PowerShell », jamais l'invite de commandes cmd.exe. Les blocs @' ... '@ sont des here-strings : le marqueur de fermeture '@ doit impérativement se trouver en début de ligne, sans espace devant, et le bloc entier se colle en une seule fois.

Objectifs pédagogiques

- Journaliser paramètres, métriques, tags et artefacts d'une exécution
- Packager un modèle avec sa signature et son environnement d'exécution
- Enregistrer plusieurs versions dans le registre et les faire evoluer par alias
- Effectuer un retour arriere sans reconstruire ni redeployer
- Servir le modèle promu en HTTP et l'appeler depuis un client

Prérequis

- TP 2 termine, pipeline fonctionnel
- Serveur MLflow démarré sur le port 5000

Installation

terminal

```
cd tp-mlops; .venv\Scripts\Activate.ps1
pip install "dvc[all]"                    # les crochets exigent les guillemets
dvc init                                  # initialise DVC dans le depot Git
New-Item -ItemType Directory -Force C:\dvcstore | Out-Null
dvc remote add -d local C:\dvcstore       # -d = stockage par defaut
git add .dvc .dvcignore; git commit -m 'chore: initialisation DVC'
```

RÉSULTAT ATTENDU

```
Requirement already satisfied (les paquets sont deja installes depuis le TP 1).
```

Déroulement

Étape 1 — Journaliser une exécution complète

Tout ce dont vous auriez besoin pour rejouer l'exécution dans six mois doit y figurer : réglages, métriques, artefacts, et surtout les liens vers le code et les données.

src/train_complet.py

```
@'
import sys, json, subprocess, mlflow, mlflow.sklearn, pandas as pd
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (f1_score, precision_score, recall_score,
                             roc_auc_score, ConfusionMatrixDisplay)
n_estimators = int(sys.argv[1]); max_depth = int(sys.argv[2])
```

TP 3 — Enregistrer modèles et artefacts dans MLflow

Durée : 3 h · Partie 3 du cours · binôme de 2

```
mlflow.set_tracking_uri('http://127.0.0.1:5000')
mlflow.set_experiment('tp3-registre')
df = pd.read_csv('data/transactions.csv')
X, y = df.drop(columns='fraude'), df['fraude']
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42)
with mlflow.start_run(run_name=f'rf-{n_estimators}-{max_depth}'):
    m = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth,
                               random_state=42).fit(Xtr, ytr)
    pred = m.predict(Xte); proba = m.predict_proba(Xte)[:, 1]
    mlflow.log_params({'n_estimators': n_estimators, 'max_depth': max_depth})
    mlflow.log_metrics({'f1': f1_score(yte, pred),
                        'precision': precision_score(yte, pred),
                        'recall': recall_score(yte, pred),
                        'roc_auc': roc_auc_score(yte, proba)})
    commit = subprocess.getoutput('git rev-parse --short HEAD')   # tracabilite
    mlflow.set_tags({'auteur': 'votre-nom', 'commit': commit,
                     'jeu_de_donnees': 'transactions v1'})
    ConfusionMatrixDisplay.from_predictions(yte, pred)            # artefact 1
    plt.savefig('reports/confusion.png', dpi=110, bbox_inches='tight')
    mlflow.log_artifact('reports/confusion.png')
    imp = dict(zip(X.columns, m.feature_importances_.round(4)))   # artefact 2
    json.dump(imp, open('reports/importances.json', 'w'), indent=2)
    mlflow.log_artifact('reports/importances.json')
    mlflow.sklearn.log_model(m, name='model', input_example=Xte[:3])  # signature
    print('F1 =', round(f1_score(yte, pred), 4))
'@ | Set-Content -Encoding UTF8 src/train_complet.py
for p in '200 8' '300 12' '500 16'; do python src/train_complet.py $p; done
```

RÉSULTAT ATTENDU

```
F1 = 0.8549
F1 = 0.8636
F1 = 0.8669

Dans l'interface, chaque execution contient l'onglet Artifacts avec la matrice de
confusion, le fichier d'importances et le dossier model.
```

Étape 2 — Inspecter le modèle packagé

Le dossier model produit par log_model contient bien plus que le modèle sérialisé : c'est ce qui permet de le servir ailleurs sans deviner le format des entrées.

terminal

```
# retrouve le dossier du modele, ou qu'il soit
$m = Get-ChildItem mlartifacts -Recurse -Filter MLmodel | Select-Object -First 1
Get-ChildItem $m.DirectoryName              # contenu du modele packagé
Get-Content $m.FullName -TotalCount 30      # les 30 premieres lignes du MLmodel
```

RÉSULTAT ATTENDU

```
MLmodel  conda.yaml  model.pkl  python_env.yaml  requirements.txt

Le fichier MLmodel decrit les flavors (python_function, sklearn) et la signature :
le nom et le type de chaque colonne attendue en entree.
```

Si ça coince : si le dossier n'est pas trouve, vérifiez le chemin affiche par MLflow dans l'onglet Artifacts de l'exécution.

TP 3 — Enregistrer modèles et artefacts dans MLflow

Durée : 3 h · Partie 3 du cours · binôme de 2

Étape 3 — Enregistrer trois versions dans le registre

Chaque exécution jugee interessante devient une version numerotee du modèle.

terminal

```
@'
import mlflow
mlflow.set_tracking_uri('http://127.0.0.1:5000')
exp = mlflow.get_experiment_by_name('tp3-registre')
runs = mlflow.search_runs([exp.experiment_id], order_by=['metrics.f1 DESC'])
for _, r in runs.head(3).iterrows():                # les 3 meilleures executions
    mv = mlflow.register_model(f"runs:/{r.run_id}/model", 'fraude')
    print('version', mv.version, '- F1', round(r['metrics.f1'], 4))
'@ | python -
```

RÉSULTAT ATTENDU

```
version 1 - F1 0.8669
version 2 - F1 0.8636
version 3 - F1 0.8549
```

Étape 4 — Piloter le cycle de vie par alias

Un alias est une étiquette mobile. Le service d'inférence ne connait que l'alias : deplacer l'alias suffit a changer le modèle servi.

terminal

```
@'
from mlflow import MlflowClient
import mlflow; mlflow.set_tracking_uri('http://127.0.0.1:5000')
c = MlflowClient()
c.set_registered_model_alias('fraude', 'staging', 2)      # v2 en pre-production
c.set_registered_model_alias('fraude', 'production', 1)   # v1 en production
c.update_model_version('fraude', 1, description='validee par le metier le 12/03')
for a in ['staging', 'production']:
    v = c.get_model_version_by_alias('fraude', a)
    print(a, '->  version', v.version)
'@ | python -
```

RÉSULTAT ATTENDU

```
staging ->  version 2
production ->  version 1
```

Étape 5 — Démontrer le retour arriere

Une regression est détectée sur la version en production. Le retour arriere ne reconstruit rien : il deplace l'alias.

terminal

```
@'
from mlflow import MlflowClient
import mlflow, pandas as pd
mlflow.set_tracking_uri('http://127.0.0.1:5000')
c = MlflowClient()
X = pd.DataFrame([{'montant': 8500, 'anciennete': 3, 'freq_24h': 9, 'nuit': 1}])
c.set_registered_model_alias('fraude', 'production', 2)   # on promeut la v2
```

TP 3 — Enregistrer modèles et artefacts dans MLflow

Durée : 3 h · Partie 3 du cours · binôme de 2

```
print('servi :', mlflow.pyfunc.load_model('models:/fraude@production').predict(X))
c.set_registered_model_alias('fraude', 'production', 1)   # retour arriere immediat
print('servi :', mlflow.pyfunc.load_model('models:/fraude@production').predict(X))
'@ | python -
```

RÉSULTAT ATTENDU

```
servi : [1]
servi : [1]

Le code du client n'a pas change d'une ligne entre les deux appels : seule
l'affectation de l'alias a bouge.
```

Étape 6 — Servir le modèle promu en HTTP

MLflow démarré lui-même un serveur d'inférence à partir du modèle packagé. Ouvrez un troisième terminal pour cette étape.

terminal 3

```
$env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"
mlflow models serve -m 'models:/fraude@production' -p 8000 --env-manager local
```

terminal 2

```
$corps = @{ dataframe_records = @(
    @{ montant = 8500; anciennete = 3; freq_24h = 9; nuit = 1 }
) } | ConvertTo-Json -Depth 5
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/invocations `
    -ContentType 'application/json' -Body $corps
```

RÉSULTAT ATTENDU

```
{"predictions": [1]}

Vous venez de servir en HTTP un modele promu depuis un registre, sans ecrire
une seule ligne de code de serveur.
```

Étape 7 — Vérifier la signature en envoyant une requête invalide

La signature enregistrée avec le modèle protégé le service contre des entrées mal formees.

terminal 2

```
$corps = @{ dataframe_records = @(
    @{ montant = 8500; anciennete = 3; freq_24h = 9; nuit = 1 }
) } | ConvertTo-Json -Depth 5
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/invocations `
    -ContentType 'application/json' -Body $corps
```

RÉSULTAT ATTENDU

```
Une erreur explicite indiquant les colonnes attendues et absentes.
Sans signature, le modele aurait echoue plus loin, avec un message incomprehensible.
```

Questions du compte rendu

1. Citez trois éléments contenus dans le dossier model en plus du fichier sérialisé, et leur utilite.

TP 3 — Enregistrer modèles et artefacts dans MLflow

Durée : 3 h · Partie 3 du cours · binôme de 2

2. Quelle est la différence entre une version et un alias ? Pourquoi le service d'inférence utilise-t-il un alias ?
3. Vous devez expliquer une decision prise il y a six mois. Quelles informations journalisées mobilisez-vous ?
4. Pourquoi journaliser le commit Git et la version du jeu de données comme tags de l'exécution ?
5. Quel est l'intérêt de la signature du modèle pour l'équipe qui developpera l'API au TP 10 ?

Livrables

- Le script src/train_complet.py
- Capture de l'onglet Artifacts d'une exécution, montrant les deux artefacts et le modèle
- Capture de l'onglet Models avec les trois versions et les deux alias
- La trace de l'appel curl réussi et celle de l'appel invalide
- Compte rendu avec les réponses aux questions

Barème

Critère	Points
Exécution complète : paramètres, métriques, tags, artefacts	5
Modèle packagé avec signature et environnement	3
Trois versions enregistrées, alias poses et documentes	4
Retour arriere démontré sans modification de code	4
Service HTTP fonctionnel et test d'entrée invalide	4